Failure (respecting a real rate limit) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `retry_backoff_practice.py` (Failure section): a real API doesn't just fail randomly — a 429 often comes with a `Retry-After` header telling you exactly how long to wait, and ignoring it makes the rate limit worse for you and everyone sharing your API key. This exercise builds the one function that reads that header, converts it to a number of seconds, caps it so a huge value can't stall you, and falls back to normal exponential backoff when it's missing or malformed. **If not:** the Build Task's `http_client.py` would fall back to blind exponential backoff even when the server explicitly says how long to wait — slower to recover, and a worse citizen on a shared API key.

Solution

```
# retry_backoff_practice.py — Failure section
from dataclasses import dataclass, field

@dataclass
class FakeResponse:
    status_code: int
    headers: dict[str, str] = field(default_factory=dict)

def decide_wait_seconds(response: FakeResponse, attempt: int) -> int:
    """Respect Retry-After if the server sent one, else fall back to backoff."""
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        try:
            return int(retry_after)
        except ValueError:
            # malformed header — fall through to backoff instead of crashing
            pass
    return 2 ** attempt

if __name__ == "__main__":
    told_to_wait = FakeResponse(status_code=429, headers={"Retry-After": "2"})
    print(decide_wait_seconds(told_to_wait, attempt=0))  # 2

    no_header = FakeResponse(status_code=429, headers={})
    print(decide_wait_seconds(no_header, attempt=3))  # 8
```

```
    bad_header = {"Retry-After": "soon"}
    malformed_header = FakeResponse(status_code=429, headers=bad_header)
    print(decide_wait_seconds(malformed_header, attempt=2))  # 4, falls back
```

Expected output:

```
2
8
4
```